

MarketSage



AI Systems Architecture Review

An MCP-native market intelligence workbench: typed finance tools, saved and auditable runs, sources that never hide what they are

Release/ship date · 12 September 2026 · Roshan Rana

Executive summary

What it is

- A Go MCP gateway exposes seven finance tools and a saved-run resource over stdio, backed by a Python FastAPI analytics core: market data, sentiment, evidence search and research briefs.
- A Next.js analyst workbench gives the same capabilities to reviewers who never touch an MCP client; DuckDB persists every dataset, run and audit event.

What it proves

- Three data modes and warnings that cannot be suppressed: seeded is deterministic and credential-free, hybrid falls back with a payload warning, live fails clearly without OpenBB.
- Every research-brief bullet carries a reference to the snapshot, evidence id or sentiment hash behind it — 12 / 12 grounded on the committed fixtures.

Why it is enterprise-ready

- Built through 13 task packs (T-001...T-013) with frozen requirements, HLD, LLD and an execution plan; one gate (``npm run check``) covers four languages plus an offline evaluation with drift checks.
- A separate dependency and vulnerability sweep (``npm run audit:deps``); CI runs the same gate on every push.

Ask of the audience: agree the pilot scope (filing evidence for the other three seeded tickers, embedding retrieval, a recorded FinBERT run, a live OpenBB provider) and the identity model that would sit in front of bearer auth.

The problem and the users

The analyst's problem

A snapshot, a price trend, what the filings and the sentiment say, then a brief — a mature workflow, spread across a quote screen, a filings reader and a spreadsheet. A chat box bolted onto the same data gives a model no structure to call, no record of what it looked at, and no way to tell live data from illustrative seed data.

The constraints

MarketSage does not execute trades, move money or provide investment advice. Every tool response must say which data mode produced it, cite the evidence behind each claim, and leave an audit row an operator can replay.

Actor	Needs	Where it is met
Analyst (primary)	Ticker in, brief out: snapshot, evidence, sentiment, caveats, in one workbench or one MCP call	Next.js workbench; `research_brief` tool
Compliance / reviewer	An audit row per call: request id, tool, status, duration, mode, warning count	DuckDB `audit_event`; `docs/security-and-ops.md`
Platform team	One local gate, no keys for the default path, optional bearer auth	`npm run check`; `MARKETSAGE_HTTP_TOKEN`
AI assistant / MCP host	Typed tools and a resource it can discover, not a chat window	7 tools, 4 prompts, 1 resource template

Solution at a glance



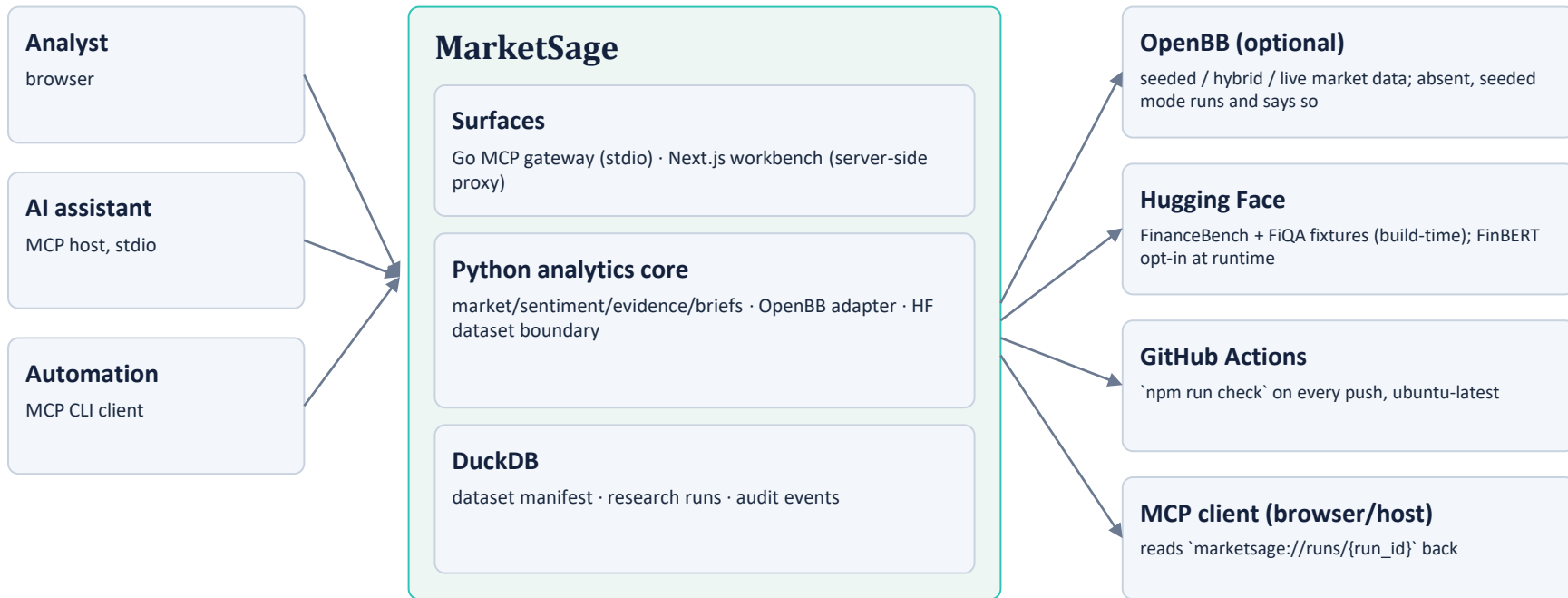
Surfaces

A Go MCP gateway over stdio, and a Next.js workbench proxying server-side, both call the same FastAPI analytics core; neither one computes anything itself.

Two data modes, one schema

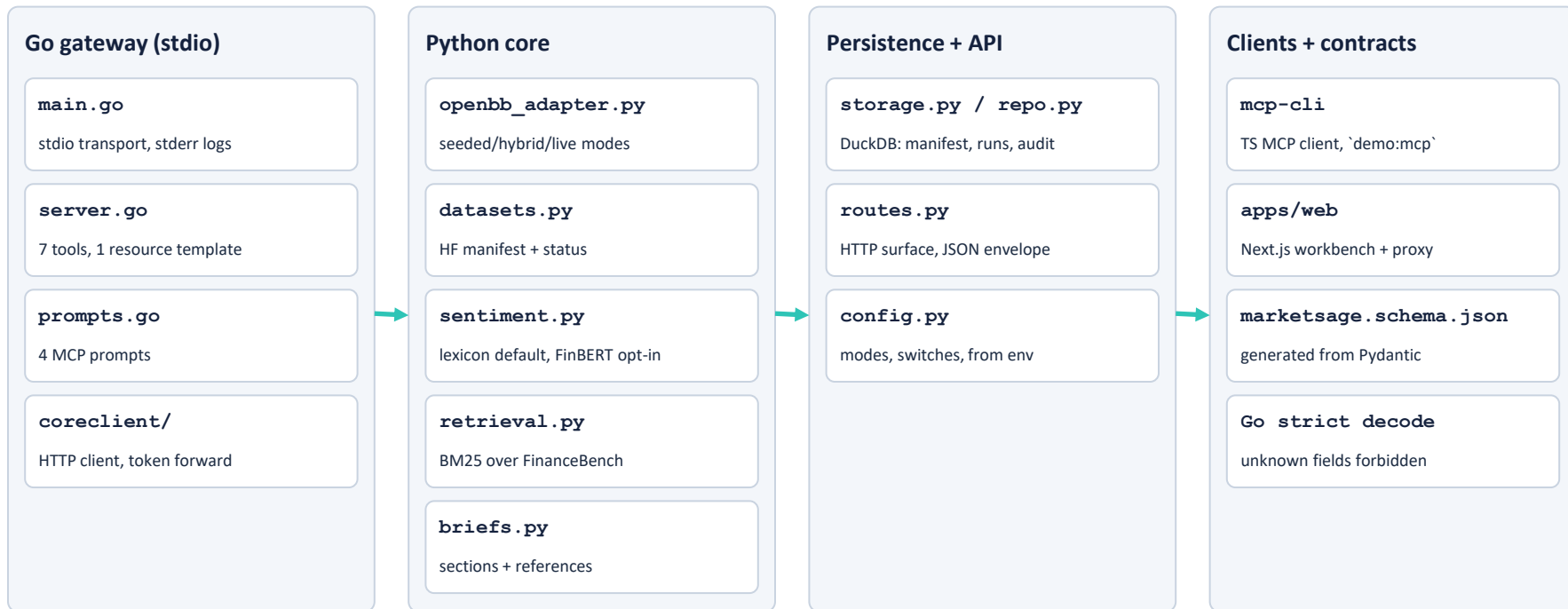
``seeded`` is deterministic and credential-free (the default). ``hybrid`` tries OpenBB and falls back with a warning in the payload. ``live`` fails clearly when the optional dependency is missing.

System context



Model downloads are off by default (MARKETSAGE_ENABLE_MODEL_DOWNLOADS enables FinBERT). Live OpenBB mode needs its optional dependency and provider configuration; the offline gate and CI never touch either.

Component architecture



One schema, generated from the Pydantic models, describes every payload; the harness validates live responses against it and exports fixtures the Go client decodes with unknown fields forbidden.

The research-brief pipeline



Failure is first-class, not an afterthought: nine named fault injections are scored 9 / 9 handled honestly — an explicit warning on degrade, a clean error and an audit row on failure, never invented data.

The core mechanism: every bullet carries a reference

One section of a real saved run (MSFT)

```
{
  "title": "Sentiment",
  "bullets": [
    "Neutral sentiment (55% confidence,
      marketsage-lexicon-v0)."
  ],
  "references": [
    "sentiment:29e7347946b40c0470c..."
  ]
}
```

What the harness scores

- every bullet's reference must resolve to a snapshot, evidence id or sentiment hash in the same payload
- a ticker with no committed evidence says so instead of borrowing another company's

12 / 12

brief claims grounded, across 5 tickers

8 / 8

responses conforming to the generated JSON Schema contract

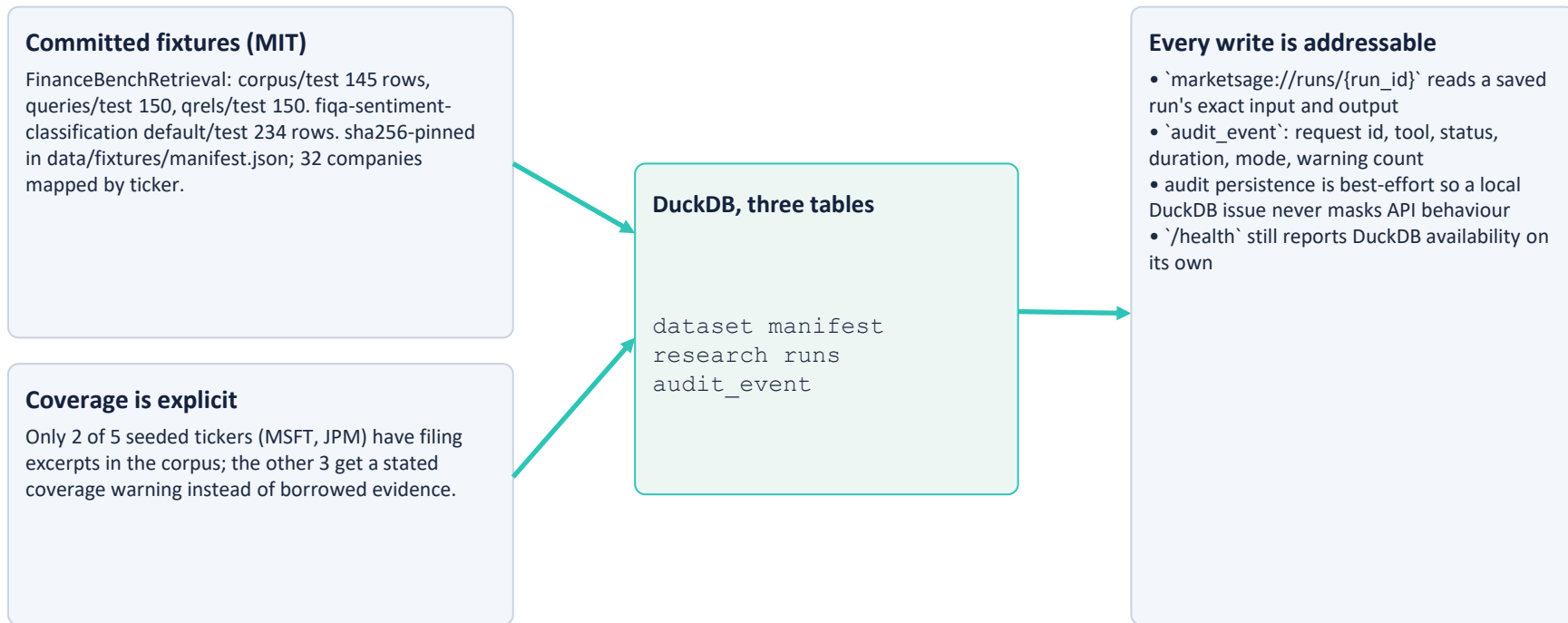
9 / 9

fault injections handled with a warning or a clean error, never invented data

Why this matters to a reviewer

The reference list is the control, and the eval suite reports grounding as a headline KPI, not a claim about prose quality.

Data architecture: fixtures, runs, audit



Seeded market data (5 illustrative tickers) is not scored by the eval harness; only the retrieval and sentiment fixtures are.

Data modes and warnings that cannot be suppressed

seeded (default)

Deterministic local data from data/seed/. No credentials, no network. What every screenshot in this deck shows.

hybrid

Tries OpenBB live data; on failure, falls back to seeded data with an explicit warning in the response payload — never a silent substitution.

live

Requires the optional OpenBB dependency and provider configuration; fails clearly, with a stated reason, when either is missing.

Model downloads: off by default

Deterministic sentiment fallback runs with no download.
`MARKETSAGE\_ENABLE\_MODEL\_DOWNLOADS=true` enables FinBERT. The offline eval harness and CI never trigger a download; FinBERT accuracy is unmeasured here and marked pending.

Protected local mode

`MARKETSAGE\_HTTP\_TOKEN` requires bearer auth on every analytics HTTP request. The Go gateway and the Next.js proxy both forward the same token server-side; `/health` reports only `http\_auth\_required`, never the token.

Design choices and why

Decision	Alternatives considered	Why this one
Go gateway + Python core + TS clients (ADR-002)	Python-only MCP server; wrap OpenBB's own MCP server directly; TypeScript-only server	Go's stdio transport discipline plus Python's OpenBB/HF ecosystem fit; a Go spike in M0 proved the boundary before deeper build
DuckDB for local persistence (ADR-003)	SQLite; PostgreSQL	Strong local analytics/parquet fit, no database server for a reviewer to stand up; schemas kept portable enough for Postgres later
AGPL-3.0-only licensing (ADR-004)	MIT with OpenBB isolated behind an external service; delay the licensing decision	OpenBB states AGPLv3; this is the safe compatibility default while OpenBB is a direct dependency
Seeded data first, live second (ADR-005)	Live-first integration	A reviewer gets a working demo with no credentials or rate limits; live OpenBB mode layers on afterward
Lexical BM25 retrieval, no vector database	Embeddings with bge-small/MiniLM + a vector store	Deterministic in CI, ticker-scoped and explainable today; embedding retrieval is one function to add, and is listed pending
Deterministic sentiment fallback as the default	FinBERT as the default classifier	No model download needed for the demo or CI; the fallback's weak 8.1% commit rate is scored and shown, not hidden

Major features

Seven MCP tools, one resource

health_check, dataset_status, market_snapshot,
price_history, sentiment_score_text, evidence_search,
research_brief; marketsage://runs/{run_id}

Four MCP prompts

equity_research, portfolio_risk_scan,
earnings_call_questions, source_quality_review; read
from marketsage-mcp --describe

Analyst workbench (Next.js)

ticker + evidence-query inputs, mode selector,
snapshot, price chart, evidence list, brief,
dataset/caveat log

Offline evaluation harness

BM25 recall, sentiment-fallback scoring, brief
grounding, contract conformance, 9 fault injections,
audit completeness, latency budget

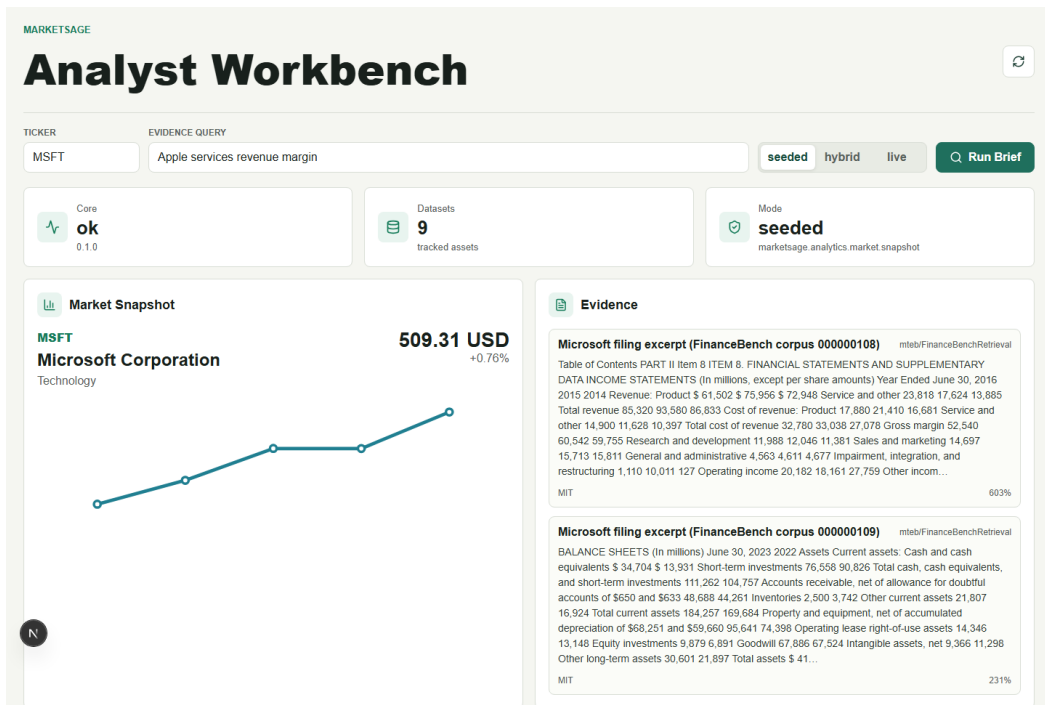
DuckDB audit trail

request id, tool, status, duration, mode and warning
count on every analytics call

Dependency and vulnerability sweeps

npm audit, uv pip check, govulncheck and a secret-text
scan, in `npm run audit:deps`

The analyst workbench: snapshot, trend and evidence



Seeded mode, no credentials. MSFT snapshot with a one-quarter close series and two FinanceBench filing excerpts scored by BM25, each captioned with its dataset and licence.

The saved run: every section keeps its references

Research Brief

MarketSage brief: MSFT

42a70eb8

Market Snapshot

- MSFT: 509.31 USD, +0.76% as of 2025-08-26T20:00:00+00:00

Sentiment

- Neutral sentiment (55% confidence, marketsage-lexicon-v0).

Evidence

- MSFT | Microsoft filing excerpt (FinanceBench corpus 000000108): Table of Contents PART II Item 8 ITEM 8. FINANCIAL STATEMENTS AND SUPPLEMENTARY DATA INCOME STATEMENTS (in millions, except per share amounts) Year Ended June 30, 2016 2015 2014 Re
- MSFT | Microsoft filing excerpt (FinanceBench corpus 000000109): BALANCE SHEETS (in millions) June 30, 2023 2022 Assets Current assets: Cash and cash equivalents \$ 34,704 \$ 13,931 Short-term investments 76,558 90,826 Total cash, cash equivalents

Caveats

- Seeded data is illustrative and not current market data.
- This output is research support, not investment advice.
- Live OpenBB mode and model downloads are optional runtime paths.

Dataset And Caveat Log

DATASET	ROWS	LICENSE	STATUS
zeroshot/twitter-financial-news-sentiment	9,543	MIT	metadata-ready
zeroshot/twitter-financial-news-sentiment	2,388	MIT	metadata-ready
TheFinAI/figa-sentiment-classification	822	MIT	metadata-ready
TheFinAI/figa-sentiment-classification	234	MIT	committed-future
TheFinAI/figa-sentiment-classification	117	MIT	metadata-ready
mtsb/FinanceBenchRetrieval	145	MIT	committed-future

- Large transcript corpus is blocked until license review; use MIT fixtures for MVP.
- Lexical bm25 retrieval over the committed FinanceBench corpus; embedding retrieval is an optional enhancement.
- Lexical bm25 retrieval over the committed FinanceBench corpus; embedding retrieval is an optional enhancement.
- Using deterministic fallback sentiment; enable model downloads for FinBERT.

The same run, readable back through `marketsage://runs/{run_id}`: a run id badge, referenced sections, and the dataset/caveat log.

Driving the gateway: tool discovery and a live call

```
PS C:\Code-Central\MarketSage> npm run demo:mcp

> tsx src/index.ts --start-analytics
MarketSage MCP tools (7)
- dataset_status: Report configured Hugging Face finance datasets, row counts, license notes, and local availability.
- evidence_search: Search the committed FinanceBench filing corpus and return citation-ready snippets.
- health_check: Report MarketSage readiness, dependency status, and seeded/live mode.
- market_snapshot: Return a normalized seeded or OpenBB-backed market snapshot for a ticker.
- price_history: Return normalized seeded or OpenBB-backed price history for a ticker.
- research_brief: Assemble a structured market research brief from market data, sentiment, and evidence.
- sentiment_score_text: Score financial text sentiment using FinBERT when enabled or a deterministic local fallback.

market_snapshot MSFT
{
  "data": { "ticker": "MSFT", "name": "Microsoft Corporation", "price": 509.31,
            "change_percent": 0.76, "source_name": "MarketSage illustrative seed data" },
  "mode": "seeded", "source": "marketsage.analytics.market.snapshot", "warnings": []
}

evidence_search "Microsoft cloud revenue growth and operating margin"
{
  "data": { "count": 2, "retrieval_mode": "lexical", "scorer": "bm25",
            "results": [
              { "id": "financebench-000000108", "ticker": "MSFT", "score": 7.3036,
                "title": "Microsoft filing excerpt (FinanceBench corpus 000000108)", "license": "MIT" },
              { "id": "financebench-000000109", "ticker": "MSFT", "score": 3.9082,
                "title": "Microsoft filing excerpt (FinanceBench corpus 000000109)", "license": "MIT" }
            ]
          },
  "warnings": ["Lexical bm25 retrieval over the committed FinanceBench corpus; embedding retrieval is an optional enhancement."]
}

resource marketsage://runs/{run_id}
[ { "uri": "marketsage://runs/5fa7a986-8598-4be9-88d6-71d18b17d2ee", "mimeType": "application/json", "text": "{ \"run_id\": \"5fa7a...\", \"output\": { \"title\": \"MarketSage brief: MSFT\", ... } }" } ]

M2 demo complete. Next milestone is the analyst workbench.
```

The TypeScript MCP client discovers all seven tools, calls `market_snapshot` and `evidence_search`, and reads the saved run back as a resource — exactly what an LLM host would see.

Every call leaves a row

```
PS C:\Code-Central\MarketSage> uv run python -c "  
import duckdb  
conn = duckdb.connect('data/local/marketsage.duckdb')  
print(conn.execute('select request_id, tool_name, status, duration_ms, mode, warning_count  
  from audit_event order by created_at desc limit 8').fetchall())  
"
```

request_id	tool_name	status	duration_ms	mode	warnings
60e23eb8-0bd2-40f8-99ff-ee8ff305d11	marketsage.analytics.briefs.research	ok	61.50	seeded	2
6396ad07-8021-4145-8932-2170ec3471cb	marketsage.analytics.market.snapshot	ok	0.60	seeded	0
78f1f663-2881-410b-94f6-ee55c0f04ba9	marketsage.analytics.evidence.search	ok	0.18	seeded	1
82a5bd22-fd61-40bf-8715-f4277a8a2c54	marketsage.analytics.market.history	ok	0.97	seeded	0
dcf462a1-fa41-45af-ac4c-614707f9dc66	marketsage.analytics.datasets	ok	168.98	seeded	1
95d6b192-3758-4bec-90d8-ab874aff36b6	marketsage.analytics.datasets	ok	215.90	seeded	1
c62c2b11-46b5-4bbb-8c13-ac62e996812b	marketsage.analytics.health	ok	2.74	seeded	0
5239a254-becc-4253-843e-ffcaef74733f	marketsage.analytics.health	ok	18.22	seeded	0

8 rows, 0 orphans. Every row carries request id, tool/source, status, duration, mode and warning count.

What you are looking at

The eight most recent audit_event rows from this session's own demo run and workbench call: request id, tool/source, status, duration, mode and warning count — 0 orphans.

Security and compliance controls

Boundary	Threat	Control in the code	Evidence
B1 client input	malformed ticker, oversized text	Pydantic validators: ticker matches <code>^[A-Z][A-Z0-9-]{0,9}\$</code> ; sentiment text bounded 1-4000 chars	models.py TickerRequest/SentimentRequest
B2 analytics HTTP API	unauthenticated access	optional bearer auth (MARKETSAGE_HTTP_TOKEN); Go gateway and Next.js proxy forward it server-side; never returned by /health	security-and-ops.md
B3 MCP stdio transport	stdout corruption breaks JSON-RPC	slog.NewJSONHandler(os.Stderr, ...) only; StdioTransport from the MCP SDK	main.go; T-011 acceptance criteria
B4 analytics responses	unaudited or silent failures	DuckDB audit_event row per call; best-effort so a local DB issue never masks API behaviour; known failures write status=error rows	storage.py, repo.py
B5 dependencies and secrets	known CVEs, leaked keys	npm audit (0 high+), uv pip check, govulncheck, and a secret-text regex scan, run in npm run audit:deps	ship-report.md, 2026-09-07
B6 data and model licensing	unlicensed reuse	source-notes.md review; sp500-earnings-transcripts blocked pending license, sec-bert-base excluded (CC-BY-SA-4.0)	docs/research/source-notes.md

Residual risk, stated plainly: the local bearer token is a demo/prototype control, not production identity management, network policy or managed secrets (docs/security-and-ops.md).

How it was built: a gated lifecycle, one validation gate



13

task packs, T-001 through T-013, each with acceptance criteria, milestone and risk tier

5

architecture decision records, each with context, options considered and consequences

45

Python tests passed (pytest), including nine named fault injections and the contract checks

11

Go tests passed, including strict decoding of the exported contract fixtures

Documents that stay frozen

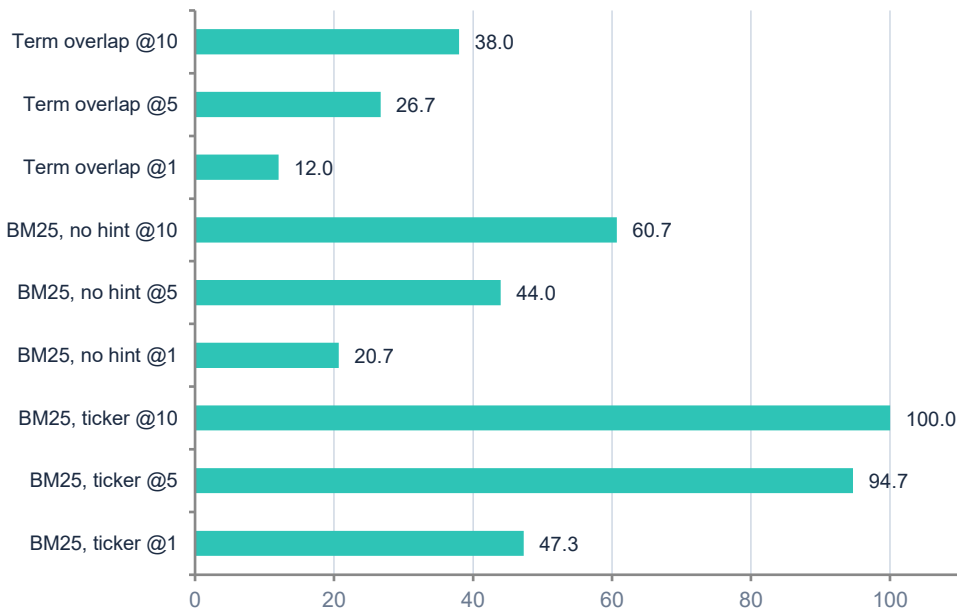
docs/design/01-requirements.md, 02-hld.md, 03-lll.md and 04-execution-plan.md are cited, not re-derived, once a task pack is in flight; docs/design/decisions.md records every ADR with its consequence.

One command validates everything

docs, ruff, pytest, go fmt/test/vet, an MCP CLI smoke, the Next.js build, a workspace check, and an offline eval with drift checks on the results card and the generated contract. CI (.github/workflows/check.yml) runs the same command on ubuntu-latest on every push.

Quality metrics from the eval harness

FinanceBench recall %, by scorer and hint (150 queries)



KPI	Value	Note
Evidence recall@5, ticker given	94.7%	no hint: 44.0%, nDCG@10 0.38
Sentiment fallback commit rate	8.1%	19/234 FiQA; 89.5% right when it fires
Brief claims grounded	12 / 12	5 tickers; 3 say so, not borrowed
Contract conformance	8 / 8	vs. the generated JSON Schema
Fault injections handled honestly	9 / 9	warning or clean error only
Audit completeness	8 / 8	0 orphaned rows
Latency budget, 8 endpoints	8 / 8 < 2s p95	5 runs; p95 63-252ms

Every figure is written by ``npm run eval`` to `metrics/headline.json`; ``npm run check`` fails if the README card or the generated contract drifts from it.

Quirks and known limitations (stated, not hidden)

Evidence coverage is 2 of 5 tickers

Only MSFT and JPM have filing excerpts in the committed corpus; the other three seeded tickers get an explicit warning, never borrowed evidence.

Recall collapses without the ticker hint

94.7% recall@5 becomes 44.0% (MRR 0.33, nDCG@10 0.38) without it; the structured hint is doing the heavy lifting, and the card says so.

The sentiment fallback is weak by design

marketsage-lexicon-v0 commits on 8.1% of FiQA test sentences (19/234); it exists so the demo runs with no download, not as a classifier.

FinBERT accuracy is unmeasured here

Needs MARKETSAGE_ENABLE_MODEL_DOWNLOADS=true and a model download; the offline harness scores the lexicon fallback only.

Embedding retrieval is not implemented

The bge-small / MiniLM path is pending; lexical BM25 is the only scorer in the eval today.

Live OpenBB data is not scored

Seeded prices are illustrative; live mode needs the optional dependency and provider configuration, and is out of the offline gate by design.

`go fmt` flags every Go file on this Windows host

A CRLF checkout artifact, not a real formatting diff (gofmt -d shows only line-ending changes); CI on ubuntu-latest is green.

Bearer auth is a demo control

MARKETSAGE_HTTP_TOKEN is local prototype protection, not production identity, network policy or managed secrets.

Cost, performance and operability

0 keys

needed for seeded mode, the demo, or the offline eval harness

8 / 8

endpoints under the 2s p95 latency budget, in-process over 5 runs

56

tests passed at ship (45 pytest + 11 Go), plus the eval harness and CI

0

high-severity findings: npm audit, uv pip check, govulncheck all clean

Operability

- demo-script.md: a five-minute walkthrough, check → demo:mcp → workbench → protected mode → eval → audit query
- SHOWCASE.md: a ten- and twenty-minute tour with commands and files
- Health endpoint reports duckdb/openbb/huggingface dependency status independently
- Protected mode is one environment variable away (MARKETSAGE_HTTP_TOKEN)

Observability and audit

- DuckDB audit_event: request id, tool, status, duration, mode, warning count, per call
- Best-effort audit writes so a local DB issue never masks API behaviour
- Every payload carries its own warnings array; nothing is silently substituted
- The saved-run resource makes every brief reproducible after the fact

Roadmap to a pilot

Now

Close the accuracy gap

- Record a FinBERT run with model downloads enabled; publish it as a recorded, not re-observed, figure
- Add embedding retrieval behind the same scorer switch, scored on the same fixture

Next

Widen evidence coverage

- Filing excerpts for the three seeded tickers that currently get a coverage warning
- Live OpenBB provider configuration and credentials for a real pilot ticker set

Then

Harden for a pilot

- An identity layer in front of MARKETSAGE_HTTP_TOKEN
- A tamper-evident audit sink, and a runbook for the on-call path

Later

Scale

- Full ticker universe instead of five seeded illustrative names
- Batch briefs and a dashboard over audit and warning rates

The first two items are the project's own stated next steps (STATE.md); nothing below changes the contracts, the audit row, or the grounding check.

Appendix A — Decision log

ID	Decision	Consequence
ADR-001	Use the full enterprise-dev-lifecycle track	Requirements, HLD, LLD, execution plan, task packs and a ship report before code merges
ADR-002	Go MCP gateway, Python analytics core, TypeScript clients	Transport discipline in Go, OpenBB/HF fit in Python; a Go spike in M0 validated the boundary first
ADR-003	DuckDB for local analytics and cache state	No database server for a reviewer; schemas kept portable enough for Postgres later
ADR-004	Default to OpenBB-compatible (AGPL-3.0-only) licensing	Avoids muddled licensing for a public repository while OpenBB is a direct dependency
ADR-005	Seeded data first, live data second	The full MCP/client workflow proven early; live OpenBB integration layered on without blocking the demo

Appendix B — Task packs and lessons

Task	Milestone	Status
T-001 Repository guardrails and validation skeleton	M0	done
T-002 ... T-004 Python core, Go gateway, TS client skeletons	M0-M1	done
T-005 ... T-009 OpenBB adapter, HF manifest, sentiment, evidence, briefs	M1-M2	done
T-010 Next.js analyst workbench	M3	done
T-011 Hardening, observability and security	M4	done
T-012 Portfolio docs and public license	M5	done
T-013 Offline evaluation harness, real evidence corpus, contracts, MCP prompts	M5	done

Lessons recorded in STATE.md and the T-013 task pack

- Phases 0-3 planning artifacts were drafted in one pass at the user's request; the hard gate before application code stayed in force regardless.
- "Not done, on purpose": FinBERT and embedding numbers need downloads the offline gate forbids; they belong in a recorded run under a bench/ path, not asserted here.
- Six hand-written evidence rows that carried a FinanceBench label without coming from it were removed once the real corpus was committed.

Appendix C — Repository map and how to run

apps/web/	Next.js workbench + server-side API proxy
clients/mcp-cli/	TypeScript MCP client (`npm run demo:mcp`)
services/	
mcp-gateway-go/	Go stdio MCP server: 7 tools, 4 prompts, 1 resource
analytics-python/	FastAPI core: market, sentiment, evidence, briefs, OpenBB adapter, HF datasets, DuckDB storage, eval
packages/contracts/	marketsage.schema.json, generated from Pydantic
data/fixtures/	FinanceBench + FiQA, MIT, sha256-pinned manifest
metrics/	headline.json, eval-latest.json, render.py
docs/design/	requirements, HLD, LLD, execution plan, decisions
docs/tasks/	13 task packs, T-001 ... T-013
docs/graph/	codebase knowledge graph (graphify): 1199 nodes, 2180 edges, 90 communities

Run it

```

npm install
npm run check           # the gate
npm run demo:mcp       # MCP client, all 7
tools
npm run eval           # the results-card
harness

# the workbench, two terminals
npm run dev:analytics
npm run dev --workspace apps/web

# protected mode
$env:MARKETSAGE_HTTP_TOKEN="<key>"
npm run dev:analytics

```

Questions

MarketSage · roshanrana/MarketSage · AGPL-3.0-only · design docs, ship report and demo script in the repository